

实验1.1

一、背景

本次实验要求编写一个**3D 占据格栅上的 A* 安全路径规划程序**，在给定三维场景、起点、目标点和障碍物的情况下，为机械臂夹爪末端规划一条安全路径。

场景来自机器人具身智能任务。视觉模型或具身智能模型可以识别桌面物体并给出目标抓取位置，但机械臂真正执行前，还需要进行路径规划，避免夹爪末端撞到桌面上的盒子、杯子、瓶子、显示器等物体。

本实验中，三维空间被离散成占据格栅：

- 每个格点表示一个可能的夹爪末端位置；
- 障碍物占据的格点不可通过；
- 基础部分只能沿 3D 6 邻域移动；
- 每次移动的代价为 1。

你需要实现 A* 搜索，并在实验报告中说明启发式函数的正确性。

二、问题描述

本实验输入一个三维网格场景，包括：

- 网格大小；
- 起点坐标；
- 目标点坐标；
- 若干个轴对齐长方体障碍物。

基础部分要求在 3D 6 邻域上找到一条从起点到目标点的最短安全路径。进阶部分可以进一步尝试让路径更平滑，更适合机械臂执行。

本实验包括：

- 基础部分：实现 3D 6 邻域 A* 最短路径搜索；
- 理论部分：证明基础部分使用的曼哈顿距离启发式函数是 admissible 和 consistent；
- 进阶部分：尝试生成更平滑、更连续的安全路径。

状态定义

一个状态是三维格点：

```
1 | (x, y, z)
```

合法坐标范围为：

```
1 | 0 <= x < X
2 | 0 <= y < Y
3 | 0 <= z < Z
```

基础部分中，相邻状态必须满足 3D 6 邻域移动，即每次只改变一个坐标，且变化量为 1。

邻域定义

对任意三维格点 $p = (x, y, z)$ ，若另一个格点 $q = (x + dx, y + dy, z + dz)$ 可以由一次合法移动到达，则称 q 是 p 的邻居。

3D 6 邻域 指只允许沿三个坐标轴方向移动一格：

```
1 | (dx, dy, dz) ∈ {
2 |   ( 1, 0, 0), (-1, 0, 0),
3 |   ( 0, 1, 0), ( 0,-1, 0),
4 |   ( 0, 0, 1), ( 0, 0,-1)
5 | }
```

也就是说，每次移动只能改变 x 、 y 、 z 中的一个坐标，且变化量为 1。这种邻接关系也称为面相邻，不允许沿边或角进行斜向移动。

26 邻域 指允许在一个动作中同时改变一个、两个或三个坐标：

```
1 | dx, dy, dz ∈ {-1, 0, 1}, 且 (dx, dy, dz) != (0, 0, 0)
```

共有 $3 * 3 * 3 - 1 = 26$ 个候选方向，包含 6 个面相邻方向、12 个边相邻方向和 8 个角相邻方向。26 邻域通常用于进阶部分生成更平滑的路径；若采用 26 邻域，移动代价可以按欧氏距离 $\sqrt{dx^2 + dy^2 + dz^2}$ 定义，也可以根据自己的平滑策略重新定义，但不要影响基础部分的 6 邻域最短路评测。

A* 评价函数

基础部分使用：

```
1 | f(n) = g(n) + h(n)
```

其中：

- $g(n)$ ：从起点到状态 n 的实际已知路径长度；
- $h(n)$ ：从状态 n 到目标点的估计距离；
- $f(n)$ ：经过状态 n 到达目标点的总代价估计。

基础部分的启发式函数使用 3D 曼哈顿距离：

```
1 | h(n) = |x - gx| + |y - gy| + |z - gz|
```

输入

详见 `input/`，共有 10 个测试文件，形式如下：

```
1 | X Y Z
2 | sx sy sz
3 | gx gy gz
4 | K
5 | x1 y1 z1 x2 y2 z2 label
6 | x1 y1 z1 x2 y2 z2 label
7 | ...
```

含义如下：

- `X Y Z`：三维网格大小；
- `sx sy sz`：起点坐标；
- `gx gy gz`：目标点坐标；
- `K`：障碍物数量；
- 每个障碍物使用闭区间 `[x1,x2] * [y1,y2] * [z1,z2]` 表示；
- `label` 是物体名称，主要用于可视化。

举例：

```
1 | 12 12 8
2 | 0 0 0
3 | 10 10 6
4 | 2
5 | 3 3 0 5 5 4 box
6 | 7 2 0 8 6 5 bottle
```

输出

基础部分结果输出到 `base_output/` 文件夹下，进阶部分结果输出到 `advanced_output/` 文件夹下。

文件命名为 `output_x.txt`，其中 `x` 表示对应的第 `x` 个输入样例。

例如对于 `input_0.txt`，输出文件名为 `output_0.txt`。

若找到路径，输出形式如下：

```
1 | path_length
2 | num_points
3 | x0 y0 z0
4 | x1 y1 z1
5 | ...
```

其中：

- `path_length`：路径长度；
- `num_points`：路径点数量；
- 第一条路径点必须是起点；
- 最后一条路径点必须是目标点。

若不存在路径，输出：

```
1 | -1
```

基础部分要求：

1. 路径点必须是整数坐标；
2. 相邻两个点必须满足 3D 6 邻域移动；
3. 路径点必须在网格范围内；
4. 路径不能经过障碍物；
5. `path_length` 必须等于 `num_points - 1`；
6. 路径长度必须等于最短路径长度。

代码框架

代码主体框架已在 `src/Astar.cpp` 和 `src/AdvancedAstar.cpp` 中给出，说明如下：

1. 已给出场景读入、障碍物占据格栅构建、边界判断、碰撞判断和输出写回等基础代码；
2. 基础部分需要在 `src/Astar.cpp` 中补全完成以下几个部分：
 - `manhattan`
 - `getNeighbors6`
 - `astar`
3. 进阶部分可在 `src/AdvancedAstar.cpp` 中继续完成路径平滑规划；
4. 给出的代码框架仅供参考，同学们可以在不改变输入输出格式的前提下自行调整实现；
5. 只需保证最后程序能够对所有输入样例产生正确输出即可。

运行示例

```
1 | python tools/run_all.py
```

该脚本会自动：

1. 处理基础部分和进阶部分程序
2. 依次处理 `input/` 下所有 `input_*.txt`
3. 将基础结果写入 `base_output/output_*.txt`
4. 将进阶结果写入 `advanced_output/output_*.txt`
5. 调用评测脚本并生成 `vis_0.png`

也可以单独进行评测：

```
1 | python tools/evaluate_base.py --input input --output base_output
2 | python tools/evaluate_advanced.py --input input --base_output base_output --
   | advanced_output advanced_output
```

三、作业要求

算法实现

1. 阅读并理解 A* 搜索算法的相关理论；
2. 编写 C/C++ 代码，实现：
 - 曼哈顿距离启发式函数；
 - 3D 6 邻域生成；
 - A* 主循环；
 - 父节点记录与路径回溯；
3. 基础部分必须输出合法且最短的路径；
4. 进阶部分可以使用 26 邻域、欧氏距离代价、转角平滑代价、shortcut smoothing 或插值等方法；
5. 本实验更关注你对 A* 搜索过程和启发式函数性质的理解。

理论证明

实验报告中需要说明：

1. 状态空间如何定义；
2. $g(n)$ 、 $h(n)$ 、 $f(n)$ 分别是什么；
3. 为什么基础部分使用曼哈顿距离；
4. 为什么曼哈顿距离是 admissible；
5. 为什么曼哈顿距离是 consistent；
6. 当 $h=0$ 时，A* 为什么退化为一致代价搜索。

证明提示：

- 在 3D 6 邻域中，每一步只能改变一个坐标，且只能改变 1；
- 从 (x, y, z) 到 (gx, gy, gz) 至少需要改变 $|x-gx|$ 次 x 坐标、 $|y-gy|$ 次 y 坐标、 $|z-gz|$ 次 z 坐标；
- 加入障碍物只可能让路径变长，不会让路径更短。

提高内容（非必做）

在完成必做部分后，可进一步尝试：

1. 使用 26 邻域生成更平滑的路径；
2. 对路径做 shortcut smoothing，删除不必要的中间点；
3. 对轨迹进行插值，使路径更连续；
4. 考虑夹爪体积、安全距离或障碍物距离惩罚项。

注意：附加策略可能改变基础最短路径性质，因此不要把加入安全距离后的结果用于基础部分自动评测。

实验报告撰写

撰写实验报告，至少包括以下内容：

1. 状态空间、动作空间和路径代价定义；
2. 基础 A* 算法流程；
3. 曼哈顿距离启发式函数的 admissible 和 consistent 证明；
4. 基础路径结果和评测结果；
5. 进阶路径规划方法；
6. 基础路径与进阶路径的对比分析。

若你自行实现了提高内容，例如：

- 障碍物膨胀；
- 安全距离；
- 平滑代价；
- 路径插值；

也欢迎在报告中单独说明。